

Python Loops

The flow of the programs written in any programming language is sequential by default. Sometimes we may need to alter the flow of the program. The execution of a specific code may need to be repeated several numbers of times.

For this purpose, The programming languages provide various types of loops which are capable of repeating some specific code several numbers of times. Consider the following diagram to understand the working of a loop statement.

Why we use loops in python?

The looping simplifies the complex problems into the easy ones. It enables us to alter the flow of the program so that instead of writing the same code again and again, we can repeat the same code for a finite number of times. For example, if we need to print the first 10 natural numbers then, instead of using the print statement 10 times, we can print inside a loop which runs up to 10 iterations.

Advantages of loops

There are the following advantages of loops in Python.

1. It provides code re-usability.
2. Using loops, we do not need to write the same code again and again.
3. Using loops, we can traverse over the elements of data structures (array or linked lists).

for loop

The **for loop in Python** is used to iterate the statements or a part of the program several times. It is frequently used to traverse the data structures like list, tuple, or dictionary.

The syntax of for loop in python is given below.

for iterating_var in sequence:

#statement(s)

The range() function

The **range()** function is used to generate the sequence of the numbers. If we pass the range(10), it will generate the numbers from 0 to 9. The syntax of the range() function is given below.

Syntax: range(start, stop, step size)

- The start represents the beginning of the iteration.
- The stop represents that the loop will iterate till stop-1. The **range(1,5)** will generate numbers 1 to 4 iterations. It is optional.
- The step size is used to skip the specific numbers from the iteration. It is optional to use. By default, the step size is 1. It is optional.

#Program to print numbers in sequence.

```
for i in range(10):  
    print(i, end = ' ')
```

Output:

0 1 2 3 4 5 6 7 8 9

#Program to print numbers in sequence defined by range.

#start from 1 and end at 9 with step 2

```
for i in range(1,10,2):  
    print(i, end = ' ')
```

Output:

1 3 5 7 9

```
#to print in the reverse order
#start from 9 end at 1 with step -2
for i in range(9,0,-2):
    print(i,end = ' ')
```

Output:

9 7 5 3 1

```
#Program to print table of given number.
n=int(input("Enter the number "))# enter 3 in prompt
for i in range(1,11):
    myvar=n*i
    print(n,"*",i,"=",myvar)
```

Output

```
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
```

```
#Program to print even number using step size in range().
n = int(input("Enter the number "))
for i in range(2,n,2):
```

```
print(i)
```

Output:

2

4

6

8

loop control statements

In Python, **loop control statements** are used to change the normal flow of a loop (like for or while). There are **three main loop control statements**:

1. break
2. continue
3. pass

break statement

The break is a keyword in python which is used to bring the program control out of the loop. The break statement breaks the loops one by one, i.e., in the case of nested loops, it breaks the inner loop first and then proceeds to outer loops. In other words, we can say that break is used to abort the current execution of the program and the control goes to the next line after the loop.

The break is commonly used in the cases where we need to break the loop for a given condition.

The syntax of the break is given below.

```
#loop statements  
    break
```

Using else statement with for loop:

Unlike other languages like C, C++, or Java, Python allows us to use the else statement with the for loop which can be executed only when all the iterations are exhausted. Here, we must notice that if the loop contains any of the break statement then the else statement will not be executed. The loop is broken due to break statement...came out of the loop

```
for i in range(0,5):  
    print(i)  
    break  
else:  
    print("for loop is exhausted")  
  
print("The loop is broken due to break statement...came out of  
the loop")
```

output:

0

*The loop is broken due to break statement...came out of the
Loop*

while loop

The Python while loop allows a part of the code to be executed until the given condition returns false. It is also known as a pre-tested loop. It can be viewed as a repeating if statement. When we don't know the number of iterations then the while loop is most effective to use.

The syntax is given below.

```
while(expression):
```

`#statements`

Here, the statements can be a single statement or a group of statements. The expression should be any valid Python expression resulting in true or false.

```
# print numbers below 10
```

```
i=0
```

```
while i<10:
```

```
    print(i)
```

```
    i=i+1
```

Output:

0

1

2

3

4

5

6

7

8

9

Using else with while loop:

Python allows us to use the else statement with the while loop also. The else block is executed when the condition given in the while statement becomes false. Like for loop, if the while loop is broken using break statement, then the else block will not be executed, and the statement present after else block will be executed. The else statement is optional to use with the while loop.

```
# The control transfer is transferred when break statement soon  
it reaches the value  
i = 0
```

```
while i < 10:  
    if(i==7):  
        break  
    else:  
        print(i)  
        i=i+1  
else:  
    print("not here...")  
print("ok here...")
```

Output:

```
0  
1  
2  
3  
4  
5  
6  
ok here...
```

```
i=1  
while(i<=5):  
    print(i)  
    i=i+1  
else:  
    print("The while loop exhausted")
```



Smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

output:

1
2
3
4
5

The while loop exhausted

continue statement

The continue statement in Python is used to bring the program control to the beginning of the loop. The continue statement skips the remaining lines of code inside the loop and start with the next iteration. It is mainly used for a particular condition inside the loop so that we can skip some specific code for a particular condition. The continue statement in Python is used to bring the program control to the beginning of the loop. The continue statement skips the remaining lines of code inside the loop and start with the next iteration. It is mainly used for a particular condition inside the loop so that we can skip some specific code for a particular condition.

Syntax

```
#loop statements
#continue

#for loop example
for i in range(1, 6):
    if(i % 2 == 0):
        continue # Skip even numbers
    print(i)
```

Output:

1
3
5

#while loop example

```
i = 0
while(i < 10):
    if(i == 5): # Skip the number 5
        i = i+1 # if this increment is not done the loop becomes
                # infinite because i=5 is not changed
        continue
    print(i)
    i = i+1
```

Output:

1
2
3
4
6
7
8
9
10

pass statement

The **pass** statement in Python is a **do-nothing placeholder** — it's used when a statement is syntactically required but you don't want to write any code yet.

```
#Inside loops or conditionals when no action is needed
for i in range(5):
    if(i == 3):
        pass # maybe do something later
    print(i)
```

Output:

0
1
2
3
4



smectechnologies

II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

GUIDING THE FUTURE WITH AI